

Arrays II & callbacks

A common array method is the `.filter()` method. When you call this method on an array, you will get back another array that contains some of the items from the original array, based on the condition you specify. Let's take an example:

```
const numbers = [9, 5, 14, 3, 11];

const numbersAboveTen = numbers.filter(function(number) {
  return number > 10;
});
console.log(numbersAboveTen); // [14, 11]

const names = ["Sam", "Alex", "Charlie"];

const result = names.find(function(name) {
  return name === "Alex";
});
console.log(result); // "Alex"
```

When you call the `.find(callback)` method on an array, you will get back **the first item** that matches the condition that you specify. If no items were found, you will get back `undefined`

`.filter()` vs `.find()`

The difference has to do with the **return type** of these 2 methods:

The `.filter()` method **always** returns an array.

The `.find()` method returns the first array item that matches the callback condition or `undefined`.

The `.map(callback)` method allows you to **transform** an array into another one. Here are some common examples:

`[4, 2, 5, 8]` transformed to `[8, 4, 10, 16]`. We doubled every item in the original array.

```
const numbers = [4, 2, 5, 8];

const doubled = numbers.map(function(number) {
  return number * 2;
});
console.log(doubled); // [8, 4, 10, 16]
```

The array `.includes(item)` method is one of the simplest array methods as it takes an `item` rather than a callback and returns `true` when that item exists in the array and `false` otherwise. Here's an example:

```
const groceries = ["Apple", "Peach", "Tomato"];

groceries.includes("Tomato"); // true
groceries.includes("Bread"); // false
```

When you have an array and you render this array to a web page (as we'll see later on in the DOM section of the course), the array will be automatically converted to a string. JavaScript will automatically invoke the `.toString()` method of the array which returns a string of the array elements separated by commas. Here's how it works:

```
const groceries = ["Apple", "Peach", "Tomato"];  
groceries.toString(); // "Apple,Peach,Tomato"
```

If you'd like to customize the glue, then you can use the `.join(glue)` method:

```
const groceries = ["Apple", "Peach", "Tomato"];  
groceries.join("; "); // "Apple; Peach; Tomato"  
groceries.join(" . "); // "Apple . Peach . Tomato"
```

[Show previous notes](#)